ELEC3104: Digital Signal Processing

Topic 4: Digital Resonator Fundamentals
MATLAB Examples

Prof. Eliathamby Ambikairajah, School of EE&T

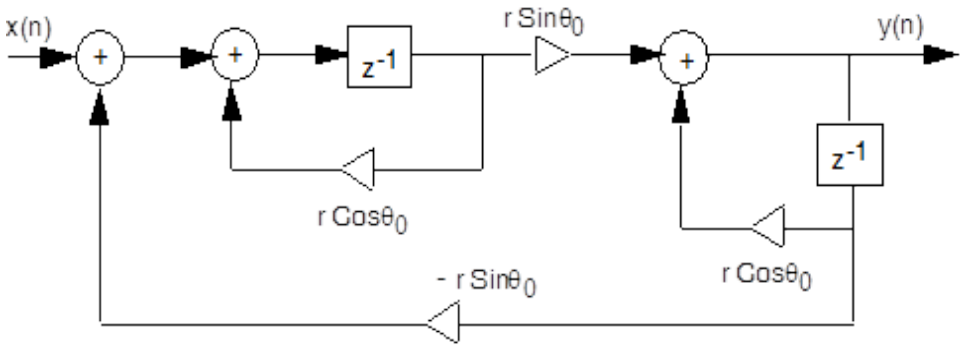
Term 3, 2026

Topic 4: Digital Resonator Fundamentals - Oscillator Implementation

Example 1

A second order system with the transfer function $H(z)$ is shown below:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{r\ sin\theta_0\ z^{-1}}{1-2\ r\ cos\theta_0\ z^{-1}+r^2z^{-2}}$$



i. Write a MATLAB code to implement the **difference equation** of system $H\ (z)$ to generate the signal $y[n]$ when $x[n] = \delta$[n] where $\theta_0 = \pi/4$, r=1. Choose the appropriate initial conditions in order to sustain oscillation.

ii. Plot the first 50 samples of y[n].
Note : DO NOT use 'impz' command.

iii. Assume that $y[n]$= sin(nθ_o), verify the value of θ_o based on the plot of $y[n]$.

iv. Obtain the pole-zero positions for the above transfer function. What do you infer from the pole zero plot?

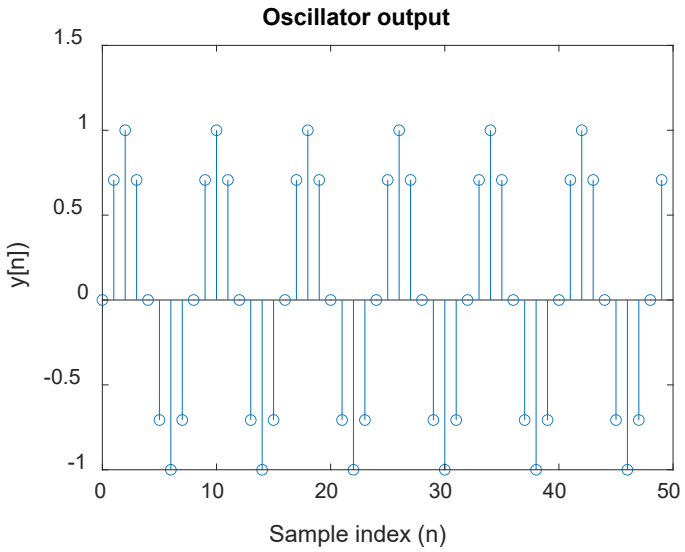
Topic 4: Digital Resonator Fundamentals - Oscillator Implementation

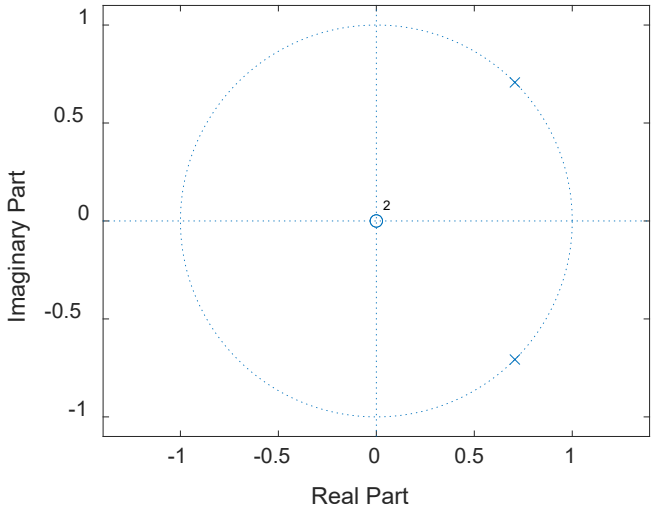
Example 1

```
% Oscillator Example
theta = pi/4;
r = 1;
N = 50; % Number of samples
n = 0:N-1;
% Impulse input x[n] = delta[n]
x = zeros(1,N);
x(1) = 1; % x[0] = 1
y = zeros(1,N); % Initialise output
% first output sample is y(1)
% initial conditions y(0) = 0; y(-1)= 0 (See lecture notes)
% y(1) in MATLAB correspond to y(0) in equation, y(0) = sin(0*theta)=0
y(1) = 0;
% y[1] = r*sin(theta0)*x[0]
y(2) = r*sin(theta0); %initial conditions y(0)=y(1)=0
for i = 3:50
    y(i) = r*sin(theta)*x(i-1) + 2*r*cos(theta)*y(i-1)-r^2*y(i-2);
end
figure
stem(n,y);
xlabel('Sample index (n)'); ylabel('y[n])');
title('Oscillator output');

% part c
% It can be checked from the plot. period of y[n] has N=8 samples
% N*theta = 2*pi .  Therefore theta = 2*pi/8 = pi/4

% part d
b = r*sin(theta); a= [1 -2*r*cos(theta) r^2];
figure
zplane(b,a);
```



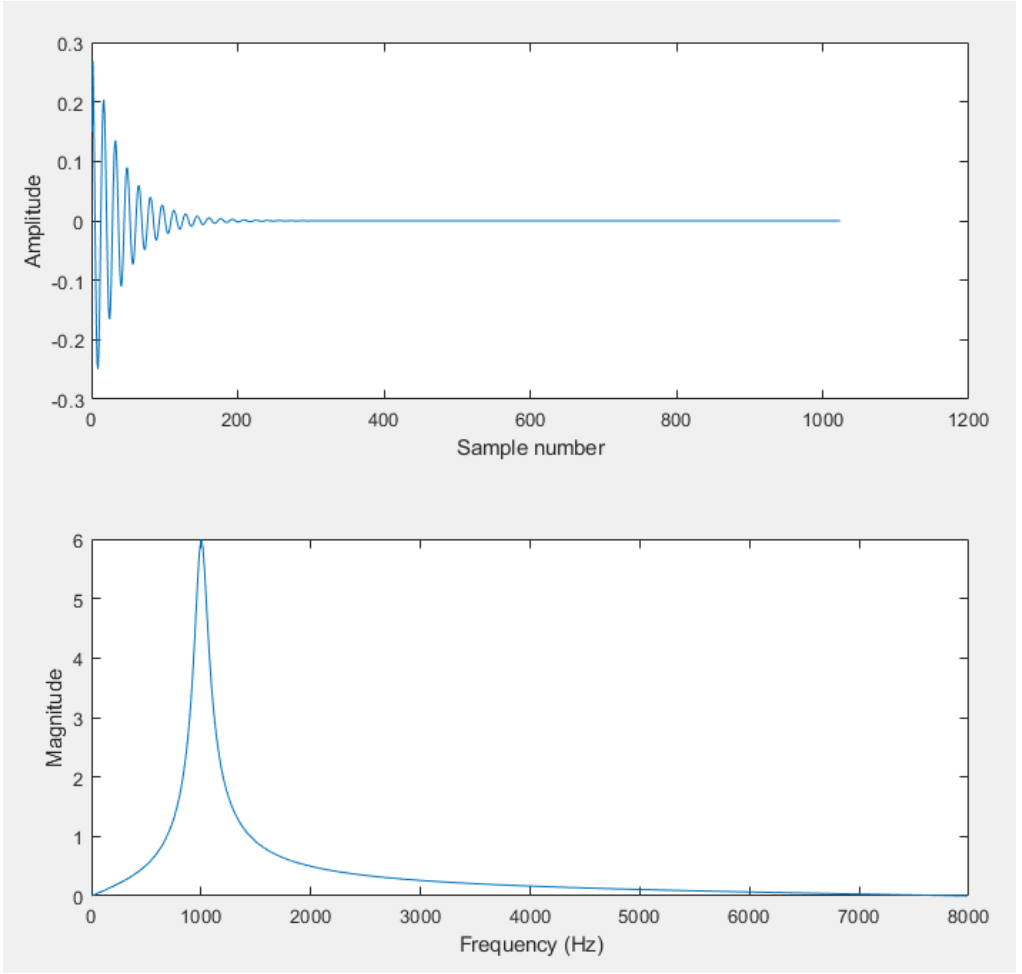


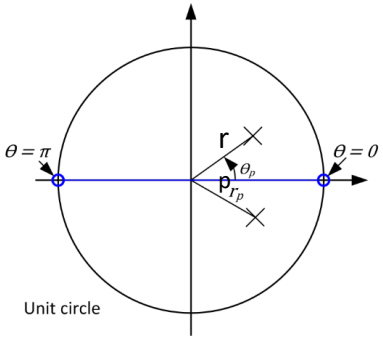
Topic 4: Digital Resonator Fundamentals - Bandpass Filter Implementation

Example 2

Design a bandpass filter with centre frequency f_p of 1000Hz , a quality factor Q_p of 7.8 and a sampling frequency of 16,000Hz according to the pole-zero diagram provided. Write a MATLAB program to compute and plot the impulse and magnitude responses of the filter.

```
fp = 1000; % resonant f r equency
fs = 16000; % sampl ing f requency
theta_p = 2*pi*(1000/ 16000 ); % digital resonant frequency
% Calculation of radius of the complex poles
% r_p = (1 - pi*(bandwidth/fs ))
r_p = 1 - (1000/(16000*7.8))* pi;
% Calculation of coefficients
b1 = 2*r_p*cos(theta_p);
b2 = r_p^2;
gp = 1 - b1 + b2; % DC gain
% There are two zeros: theta= 0 and theta= pi
a0 = 1; a1 = 0; a2 = -1; % H(z) = 1-z^-2
% Impulse response
impulse= gp * impz ( [a0 a1 a2], [1 -b1 b2],1024) ;
len = length(impulse);
resolution= 16000/len;
freq_x = (0:len-1)*resolution;
mag_p = abs(fft(impulse));
figure
subplot 211
plot(impulse);
xlabel('Sample number');
ylabel('Amplitude');
subplot 212
plot(freq_x(1:len/2), mag_p(1:(len/2)));
xlabel('Frequency (Hz)'); ylabel('Magnitude');
```





ELEC3104: Digital Signal Processing

Topic 4: Digital Resonator Fundamentals
MATLAB Exercises

Prof. Eliathamby Ambikairajah, School of EE&T

Term 3, 2026

Exercises – Digital Resonator Fundamentals (Topic 4)

Exercise 1 - Digital Oscillator

A digital oscillator is given by the following difference equation: $y[n] = b_1\, y[n-1] - b_2\, y[n-2]$

Where the filter coefficients $b_1 = 2\cos(\theta_0)$, $b_2 = 1$ and θ_0 is the oscillator frequency. The initial conditions are: $y[0] = A$; $y[-1] = A\cos(\theta_0)$; where A is the amplitude of oscillation (You may choose the value of A)

Using MATLAB, write a code to implement the above difference equation and show that the oscillator is oscillating at the appropriate frequency , θ_0 , of your choice.
Discuss your results with your lab demonstrator.

Exercise 2: Two-Pole Resonator

Design a two-pole resonator with a peak gain of unity at $f_0 = 500$ Hz and the bandwidth $(\Delta f) = 32$ Hz, operating at the sampling frequency $(fs) = 10$ kHz and show it's transfer function is given by

$$H(z) = \frac{0.0062}{1 - 1.8829z^{-1} + 0.98z^{-2}}$$

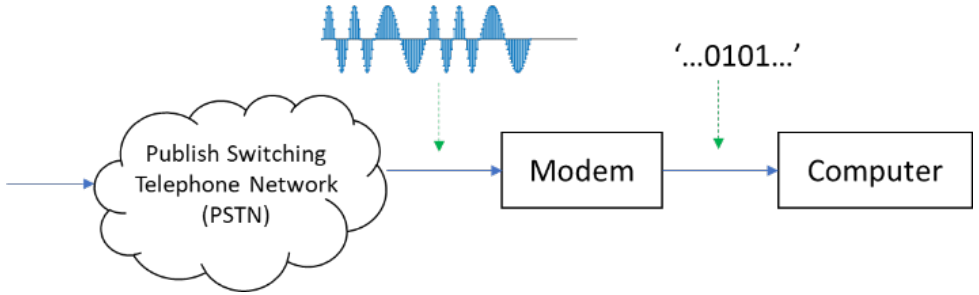
Write a MATLAB code to:

a. Plot the pole-zero diagram of the filter $H(z)$.
b. Plot the magnitude response of the above filter $H(z)$ and compute the bandwidth of the filter and see if it matches 32Hz.
c. Plot the impulse response of the filter and discuss your results with the lab demonstrator.

Exercises – Digital transmission of data

Exercise 3 - Digital data transmission application

In ancient times, modems worked by transmitting digital data through telephone lines. Specifically, during transmission modems converted a stream of 1's and 0's into an audio signal where 1's were represented by a particular frequency and 0's by another frequency. A modem acting as a receiver would convert the audio signal back into a stream of 1's and 0's. You are required to design and implement a system in MATLAB (as a MATLAB function) that will accept such audio signals (supplied to you as wav files – 'bit˙sequence1.wav', 'bit˙sequence2.wav', 'bit˙sequence3.wav' and 'bit˙sequence4.wav' located in Moodle folder Resources/Wav files for DSP Labs) as input and convert them into sequences of 1's and 0's accurately.



File	Decoded Bit Sequence
bit_sequence1.wav	
bit_sequence2.wav	
bit_sequence3.wav	
bit_sequence4.wav	

You have been given the following information about the audio signal:

1. Each symbol (i.e., 0 or 1) is represented by a sinusoid of a single frequency.
2. Each transmission begins with 0101.
3. The transmission rate is 20 bits/second

Exercises – Digital Resonator Fundamentals (Topic 4)

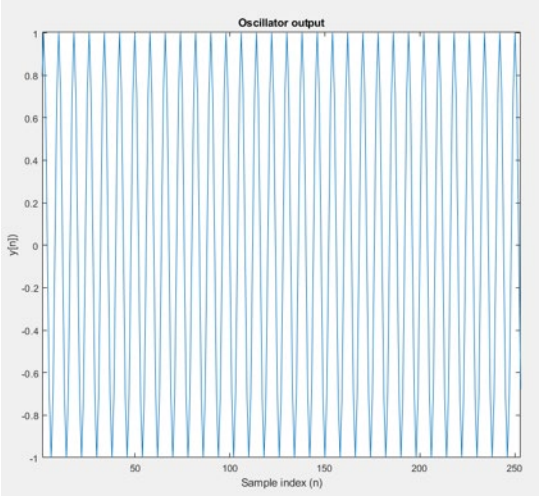
Exercise 1 - Digital Oscillator

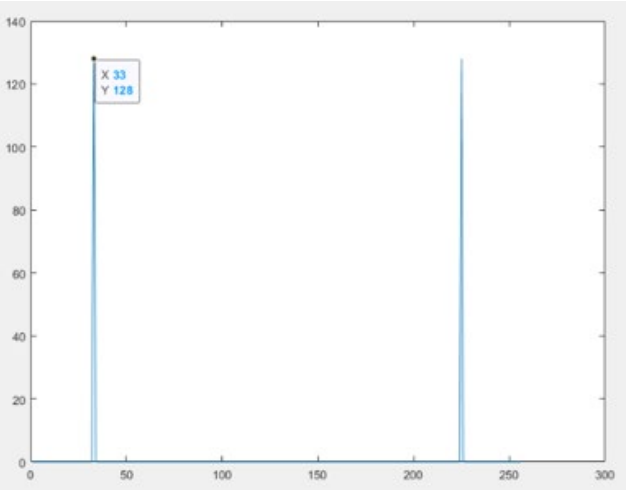
A digital oscillator is given by the following difference equation: $y[n] = b_1\, y[n-1] - b_2\, y[n-2]$

Where the filter coefficients $b_1 = 2\cos(\theta_0)$, $b_2 = 1$ and θ_0 is the oscillator frequency. The initial conditions are: $y[0] = A$; $y[-1] = A\cos(\theta_0)$; where A is the amplitude of oscillation (You may choose the value of A)

Using MATLAB, write a code to implement the above difference equation and show that the oscillator is oscillating at the appropriate frequency , θ_0 , of your choice. Discuss your results with your lab demonstrator.

```
% Oscillator Exercise
numberofsamples = 256;
theta = (pi)/4; % chosen frequency
A = 1; % Amplitude of oscilation
% first output sample is y(1)
% initial conditions y(0) = A; y(-1)= A*cos(theta)
b1 = 2*cos(theta);  b2 = 1;
% Since y(-1) and y(0) can't be represented in Matlab
% y(2) corresponds to A and y(1) corresponds to A*cos(theta)
y(1) = A*cos(theta);  y(2) = A;
for n = 3:numberofsamples
    y(n) = b1*y(n-1) - b2*y(n-2);
end
figure
plot((1:numberofsamples),y);  xlabel('Sample index (n)'); ylabel('y[n])');
title('Oscillator output');
% To verify the frequency of oscillation
% Take the FFT and find the peak is in 33rd time sample
% 33 - 1(DC comp) = 32nd sample -> Pi/4 (128/4) -> Verified
figure
plot(abs(fft(y)));
```





Exercises – Digital Resonator Fundamentals (Topic 4)

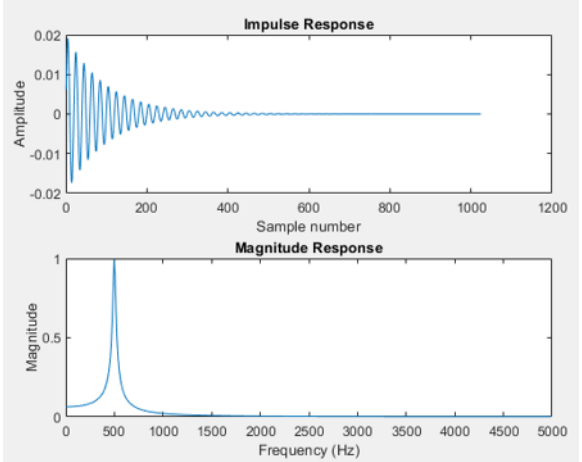
Exercise 2: Two-Pole Resonator

Design a two-pole resonator with a peak gain of unity at $f_0 = 500$ Hz and the bandwidth $(\Delta f) = 32$ Hz, operating at the sampling frequency $(fs) = 10$ kHz and show it's transfer function is given by

$$H(z) = \frac{0.0062}{1-1.8829z^{-1}+0.98z^{-2}}$$

Write a MATLAB code to:

a. Plot the pole-zero diagram of the filter $H(z)$.
b. Plot the magnitude response of the above filter $H(z)$ and compute the bandwidth of the filter and see if it matches 32Hz.
c. Plot the impulse response of the filter and discuss your results with the lab demonstrator.

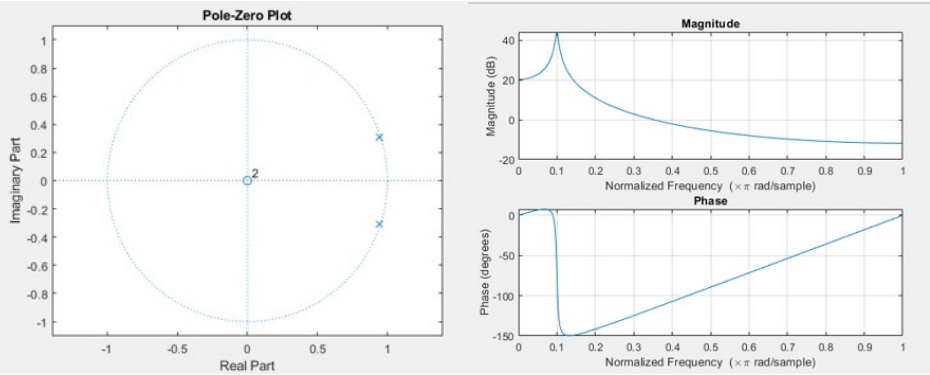


- The resonant frequency $\theta_0 = \frac{2\pi(500)}{10000} = \frac{\pi}{10}$;
- The 3-dB bandwidth $\Delta\theta = \frac{2\pi(32)}{10000} = 0.0064\pi$.
- The pole radius (r) = 1- 0.5 $\Delta\theta$ = 1-0.0032 π = 0.9899

$$H(z) = \frac{k_0 z^2}{z^2-2r\cos\theta_0 z+r^2} = \frac{k_0 z^2}{z^2-1.8829z+0.98}$$

- For a peak gain of unity, we choose $k_0 = (1-r^2)\sin(\theta_0) = 0.0062$

$$\therefore H(z) = \frac{0.0062z^2}{z^2-1.8829z+0.9800} = \frac{0.0062}{1-1.88292z^{-1}+0.9800z^{-2}}$$



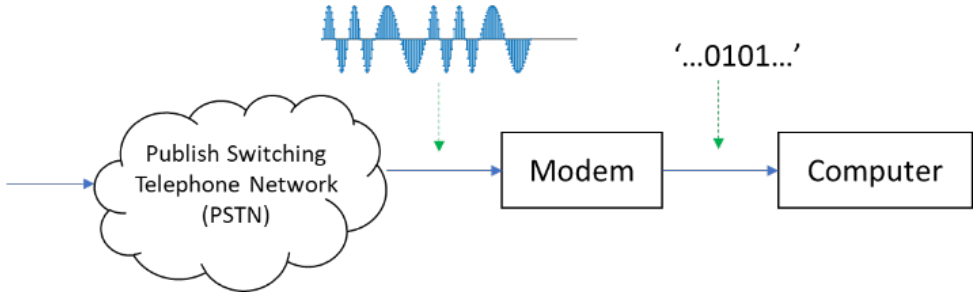

```
fp = 500; % resonant frequency
fs = 10000; % sampling frequency
bw_hertz = 32;
theta_p = 2*pi *(500/10000); % digital resonant frequency
bw_rad = 2*pi*(bw_hertz)/fs;
% Calculation of radius of the complex poles
r_p = 1 - 0.5* bw_rad;
% Calculation of coefficients
b1 = 2*r_p*cos(theta_p);
b2 = r_p^2;
k0 = (1-r_p^2) * sin(theta_p); % Peak gain of unity
a0 = 1; a1 = 0; a2 = 0;
figure
zplane([1], [1 -b1 b2]);
figure
freqz([1], [1 -b1 b2]);
```

```
% Impulse response
impulse = k0 * impz([a0 a1 a2], [1 -b1 b2],1024);
len = length(impulse);
resolution = fs/len;
freq_x = (0:len-1)*resolution;
mag_p = abs(fft(impulse));
figure
subplot 211
plot(impulse);
xlabel('Sample number');
ylabel('Amplitude');
title('Impulse Response');
subplot 212
plot(freq_x(1:len/2), mag_p(1:(len/2)));
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Magnitude Response');
```

Exercises – Digital transmission of data

Exercise 3 - Digital data transmission application

In ancient times, modems worked by transmitting digital data through telephone lines. Specifically, during transmission modems converted a stream of 1's and 0's into an audio signal where 1's were represented by a particular frequency and 0's by another frequency. A modem acting as a receiver would convert the audio signal back into a stream of 1's and 0's. You are required to design and implement a system in MATLAB (as a MATLAB function) that will accept such audio signals (supplied to you as wav files – 'bit_sequence1.wav', 'bit_sequence2.wav', 'bit_sequence3.wav' and 'bit_sequence4.wav' located in Moodle folder Resources/Wav files for DSP Labs) as input and convert them into sequences of 1's and 0's accurately.



File	Decoded Bit Sequence
bit_sequence1.wav	
bit_sequence2.wav	
bit_sequence3.wav	
bit_sequence4.wav	

You have been given the following information about the audio signal:

1. Each symbol (i.e., 0 or 1) is represented by a sinusoid of a single frequency.
2. Each transmission begins with 0101.
3. The transmission rate is 20 bits/second

Exercise 3 – Digital Data Transmission Application

```
[x,fs]=audioread('bit_sequence1.wav');  % read the sequence  1
NFFT=1024; % number of fft points
freq_pos=0:NFFT-1; % frequency positions
freqHz=fs*freq_pos/NFFT; % frequency in Hz
freqHz=freqHz(1:NFFT/2); % 0 - fs/2 in Hz
OneBitLen=fs/20; % 20 bits per second
NoOfBits=length(x)/OneBitLen; % number of bits
BitSeq=''; % initialization
% loop through all the bits
for i=1:NoOfBits
        OneBitSegment=x((i-1)*OneBitLen+1:i*OneBitLen); % acquire each bit segment (800 points)
        OneBitSegmentFFT=abs(fft(OneBitSegment,NFFT)); % find magnitude spectrum of bit segment
        OneBitSegmentFFT=OneBitSegmentFFT(1:NFFT/2); % retain magnitude spectrum upto fs/2
        % i=1 => first bit.  Each transmission begins with 0101, hence first bit = 0
        if i==1    % Determine frequency of '0' bit
                Bit0Freq=freqHz(find(OneBitSegmentFFT==max(OneBitSegmentFFT))); % find command will find the index where max occurs
                BitSeq=[BitSeq '0']; % 0 inserted to bit sequence
        % i=2 => second bit.  Each transmission begins with 0101, hence second bit = 1
        elseif i==2     % Determine frequency of '1' bit
                Bit1Freq=freqHz(find(OneBitSegmentFFT==max(OneBitSegmentFFT)));
                BitSeq=[BitSeq '1']; % 1 inserted to bit sequence
        elseif i>2      % now check all the remaining bits to match frequency of '0' or '1'
                BitFreq=freqHz(find(OneBitSegmentFFT==max(OneBitSegmentFFT)));
                switch BitFreq    % instead of using 'if' command use 'switch' command
                        case Bit1Freq
                                BitSeq=[BitSeq '1'];
                        case Bit0Freq
                                BitSeq=[BitSeq '0'];
                end
        end
end
BitSeq  % print bit sequence
% repeat this code for all the other sequence
```